

业务全景与端到端主线

gomall 一次购物穿过的 15 个域——怎么连 / 怎么读 / 卡在哪

gomall · 业务侧 deck

业务全景：gomall 到底在做什么

系统架构：分层拓扑与领域地图

端到端主线：一次成功购物的”黄金路径”

贯穿全局的 4 个硬骨头

订单状态机：把 7 跳串成一条可查的线

怎么读这套 deck：路线与角色

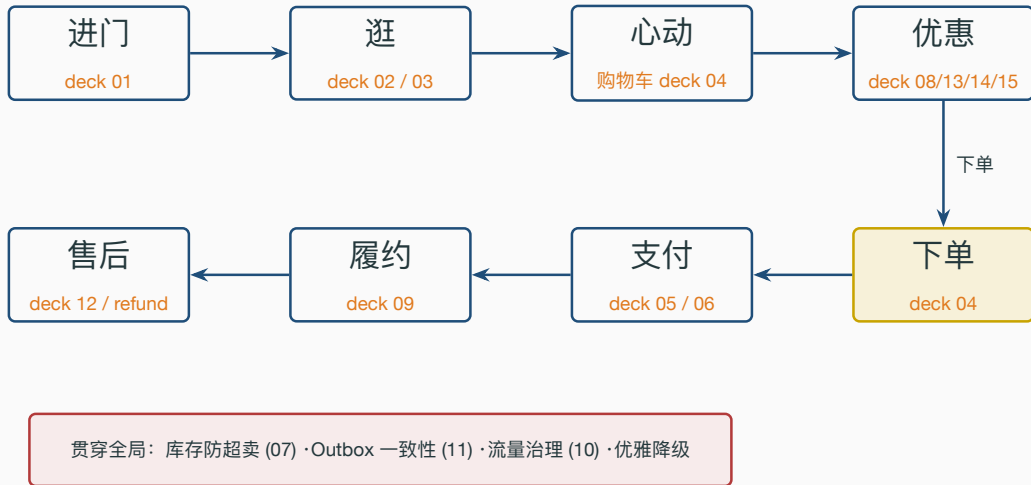
业务全景：gomall 到底在做什么

先认人：一笔订单背后站着 5 个角色

- **C 端用户**：进门 → 逛 → 下单 → 收货。只关心快、对、不丢。
- **商家 (Boss)**：订单是”我能不能发货”的唯一源头；货卖出去钱要到账。
- **运营**：办活动拉量——优惠券 / 满减 / 秒杀 / 拼团 / 预售，预算不能被刷穿。
- **客服**：用户来问”我的钱去哪了 / 货到哪了”，得有一条能查的状态线。
- **SRE**：大促洪峰别把库存卖超、别把数据库打挂，挂了要能降级。

这套 deck 按**业务域**垂直切成 15 份，每份讲透一个域。
本份是横向的”地图”：把 15 个域串成用户真实走过的一条线。

业务全景图：一次购物穿过的 8 个环节



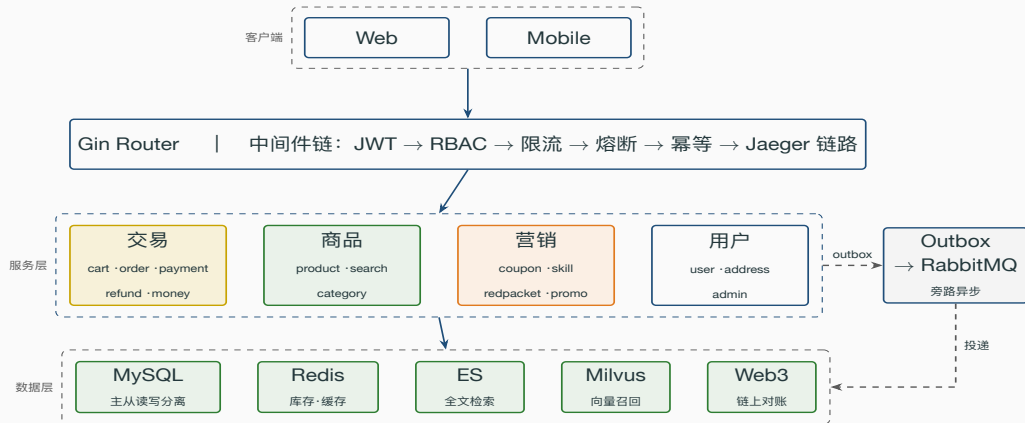
15 份 deck 索引：域 → 在哪深讲

#	业务域	一句话
01	用户与鉴权	注册 / 登录 / JWT 双 token / RBAC
02	商品展示	list / show / Cache Aside + HTTP cache
03	商品搜索	LIKE → ES → ES+Milvus 混合召回
04	购物车 → 下单	库存预扣 + DB+outbox 同事务（主线核心）
05 / 06	支付（法币 / Web3）	余额加密 + 熔断幂等 / Escrow + 链上对账
07	库存防超卖	两桶 Lua (available / reserved) + Saga 回滚
08 / 13	营销 / 满减引擎	券 / 秒杀 / 红包 / 满减最优解
09	订单生命周期	状态机 + 延迟关单双保险
10 / 11	流量治理 / 一致性	限流熔断削峰 / Outbox + Saga
12	商家后台	merchant 三层鉴权 + 可观测性
14 / 15	拼团 / 预售	成团状态机 / 定金尾款两段付

docs/slides/README.md §deck 索引；本表为 15 份现有 deck 的横向汇总

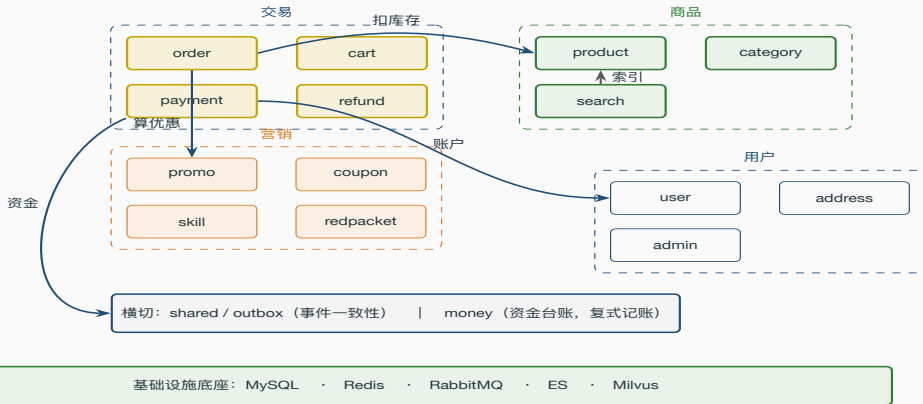
系统架构：分层拓扑与领域地图

gomall 整体架构：一条请求从客户端落到数据层



核心链路同步落库收钱，事件经 Outbox 旁路异步散给搜索 / 统计 / 履约，主线不被下游拖慢。

DDD 领域地图：22 个域按 bounded context 垂直切片



按域垂直切片，跨域只经属主领域的服务方法调用；product 与 search 已斩环（search 单向订阅 product 索引事件）。

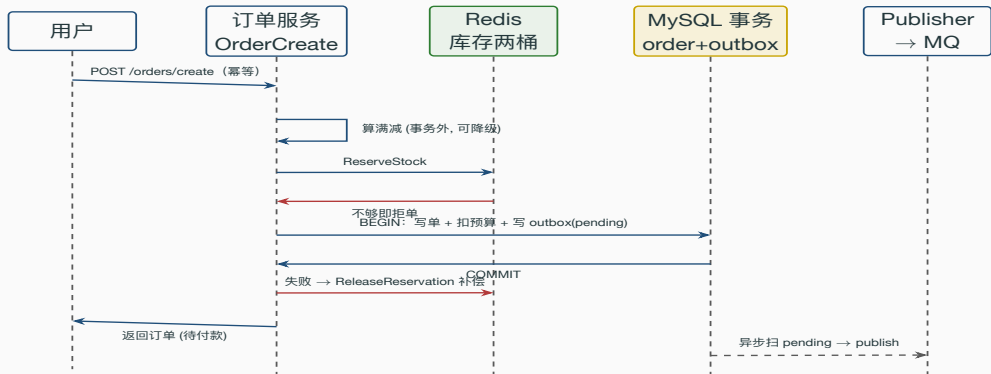
端到端主线：一次成功购物的”黄金路径”

黄金路径：7 跳，每一跳都可能掉链子

1. 登录——拿到 JWT，后续每个请求带着它过鉴权中间件。
2. 看商品——详情走 Cache Aside，热点不打穿 DB。
3. 加购 / 选地址——没地址下不了单，默认地址兜底。
4. 下单——算优惠 → 预扣库存 → 一个事务落订单 + 事件。
5. 支付——扣余额 + 改状态 + 真正消库存，同事务发”已支付”事件。
6. 发货 / 收货——状态机推进，每一步只允许合法迁移。
7. 结清——订单 Completed；中途不付钱则 15 分钟自动关单。

这条线横跨 user / product / cart / promo / order / inventory / payment 七个包，
任何一跳的失败都不能让用户”钱货两失”——这是整套设计的第一性原则。

端到端时序：下单这一跳的内部（先预扣、再事务、后通知）



internal/order/service.go:110 Reserve :117 Tx :145 outbox :159 释放补偿:172 延迟关单

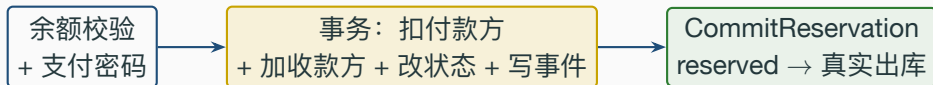
下单中枢的关键代码：顺序就是设计

Listing 1: internal/order/service.go OrderCreate 主体

```
1 // 1) 满减在事务外算：纯读路径，失败降级为无折扣，绝不阻断下单
2 promoApply, promoErr := s.promo.CalculateBestDiscount(ctx, cartItems)
3
4 // 2) Redis 预扣库存：available -> reserved，失败直接拒单，连 DB 都不碰
5 if err = cache.ReserveStock(ctx, req.ProductID, int64(req.Num)); err != nil {
6     return nil, err
7 }
8
9 // 3) 同一个事务：写订单 + 扣满减预算 + 写 outbox 事件（原子一致）
10 err = dao.NewDBClient(ctx).Transaction(func(tx *gorm.DB) error {
11     NewOrderDaoByDB(tx).CreateOrder(order)
12     s.promo.ApplyDiscountInTx(tx, order.ID, order.PromoRuleID, order.PromoDiscountCents)
13     return outbox.NewOutboxDaoByDB(tx).Insert("order", "OrderCreated", ...)
14 })
15 if err != nil { // 4) 事务失败：把第 2 步预扣的库存还回去
16     cache.ReleaseReservation(ctx, req.ProductID, int64(req.Num))
17 }
```

支付这一跳：钱和库存在这里才真正落定

- 下单时库存只是 **reserved** (预占)，钱还没动——订单状态 WaitPay。
- 支付成功才在一个事务里：扣用户余额 → 加商家余额 → 改状态 WaitShip → 写 order.paid 事件。
- 事务提交后同步 CommitReservation：把 reserved 真正消掉，库存数字才最终减少。



internal/payment/service.go:36 PayDown; repository/cache/inventory.go:98 CommitReservation

支付事务体：钱、货、状态、事件一把梭

Listing 2: internal/payment/service.go PayDown 事务体（节选）

```
1 err = orderpkg.NewOrderDao(ctx).Transaction(func(tx *gorm.DB) error {
2     order, _ := orderpkg.NewOrderDaoByDB(tx).GetOrderById(req.OrderId, uId)
3     if order.Type != consts.OrderWaitPay { // 幂等：非待付款拒绝重复支付
4         return errors.New("订单状态非未支付")
5     }
6     // 扣买家余额 (AES 密文解密校验支付密码) + 加商家余额
7     userDao.UpdateUserById(uId, buyer) // 买家 - totalMoney
8     userDao.UpdateUserById(bossID, boss) // 商家 + totalMoney
9     order.Type = consts.OrderWaitShip // 状态推进：待付款 -> 待发货
10    orderpkg.NewOrderDaoByDB(tx).UpdateOrderById(req.OrderId, uId, order)
11    return outbox.NewOutboxDaoByDB(tx).Insert( // order.paid 事件同事务落库
12        "order", "OrderPaid", "order.paid", order.ID, events.OrderPaid{...})
13 })
14 // 事务提交后：把 Redis reserved 桶真正消掉，库存数字最终减少
15 cache.CommitReservation(ctx, paidProductID, int64(paidNum))
```

贯穿全局的 4 个硬骨头

硬骨头 1：库存防超卖——为什么不用 UPDATE WHERE stock>0

- 高并发下行锁排队 + 尾延迟爆炸；下单链路最怕在事务里等锁。
- gomall 把库存搬到 Redis，用 **两个桶**：available（可卖）/ reserved（已占）。
- 预扣 / 提交 / 释放都是 **单个 Lua 脚本原子完成**，一次 RTT，天然不超卖。

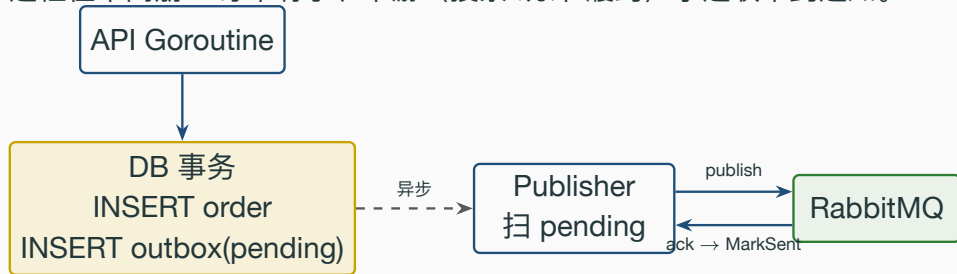
Listing 3: reserveScript: available 与 reserved 两个 key 原子流转

```
1 -- KEYS[1]=available 桶 KEYS[2]=reserved 桶 ARGV[1]=n
2 local avail = redis.call('GET', KEYS[1])
3 if avail == false then return -2 end -- 库存未初始化
4 if tonumber(avail) < tonumber(ARGV[1]) then
5     return -1 -- 不够，拒单
6 end
7 redis.call('DECRBY', KEYS[1], ARGV[1]) -- available -= n
8 redis.call('INCRBY', KEYS[2], ARGV[1]) -- reserved += n
9 return 1
```

repository/cache/inventory.go:32 reserveScript ·:78 ReserveStock 封装·:98/:113 Commit/Release (deck 07 详解)

硬骨头 2: Outbox ——解决” 订单存了但消息丢了”

双写问题：先 INSERT order 再 publish MQ，两个动作不在一个事务里。
进程在中间崩 = 订单有了、下游（搜索/统计/履约）永远收不到通知。



订单与事件**同生共死**写进一个事务；投递交给独立 publisher 重试。
至少一次投递 + 下游幂等消费 = 事件不丢。

硬骨头 3：优雅降级——核心稳如磐石，边缘随时可弃

能力	挂了会怎样	gomall 的选择
满减计算	算不出折扣	降级为 无折扣 ，照样下单
满减预算	预算抢光	事务内 改写为无折扣 ，不报错
RabbitMQ	延迟关单不可用	Redis ZSet 定时扫描 兜底关单
ES / Milvus	搜索退化	退回 DB LIKE 路径

判断标准只有一句：它是不是“用户付钱拿货”的必经环节？
是 → 进事务、保原子；不是 → 失败降级，绝不阻断 happy path。

internal/order/service.go:78 满减降级 ·:126 预算耗尽降级；cmd/main.go:69-102 ES/Milvus 静默跳过

硬骨头 4：流量治理——大促洪峰怎么扛

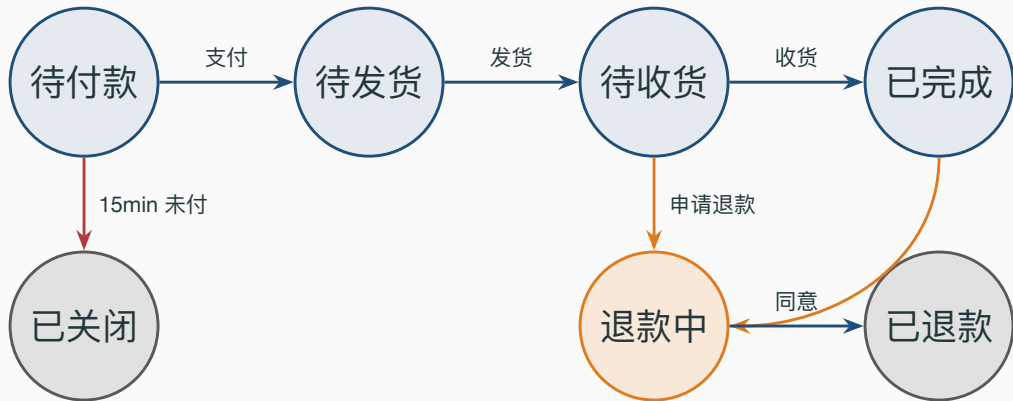
- **限流**：每 IP 令牌桶 100 RPS / 突发 200，挡爬虫与脚本。
- **幂等**：下单走 `middleware.Idempotency()`，用户狂点也只成一单。
- **削峰**：`/orders/enqueue` 投 MQ，前端拿 ticket 轮询，把洪峰摊平。

压测实证：幂等中间件下 **755K 次重放请求** → **恰好 1 笔订单**，首单成功后续请求由 Lua 读 done 态返回，**50K RPS / p95 2.33ms**。

`routes/router.go:41` 令牌桶；`internal/order/routes.go` 幂等 +enqueue；`stressTest/REPORT.md` § 幂等中间件

订单状态机：把 7 跳串成一条可查的
线

订单不是一个布尔值，是一台状态机



每一次状态变更都走 CanTransit 校验，非法迁移直接报错——
“已完成”的订单不可能跳回“待付款”，客服查到的状态永远自治。

谁来推状态：用户 / 商家 / 系统三方驱动

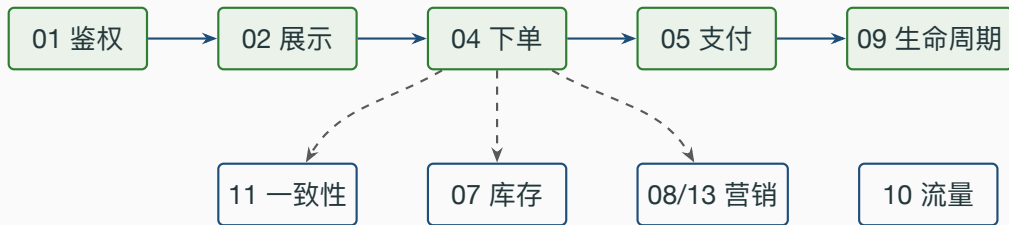
迁移	触发方	入口	权限
待付款 → 待发货	用户	支付成功	登录
待发货 → 待收货	商家	ShipOrder	admin RBAC
待收货 → 已完成	用户	ConfirmReceive	登录
待付款 → 已关闭	系统	延迟关单	MQ / Cron
* → 退款中	用户/客服	refund	登录

”系统”这一行最容易被忽略：没人点按钮，靠 RabbitMQ 延迟消息 + Redis ZSet 扫描**双保险**把僵尸订单关掉、库存还回。

internal/order/shipping.go:36 ShipOrder ·:73 ConfirmReceive; internal/order/cancel.go:19 CancelUnpaidOrder

怎么读这套 deck：路线与角色

阅读路线图：主干一条线，支线随用随查



(下单为枢纽，需要深究时下钻到支线)

第一遍：沿绿色主干 01→02→04→05→09 走一遍，建立全局。

第二遍：哪块想深究，从 04 下钻到 07 / 11 / 08 / 10。

各 deck 末页” 配套 deck” 指向相邻域

按角色取你最关心的几份

- 后端新人：01 → 02 → 04，把”一个请求怎么从路由走到 DB”吃透。
- 做交易的：04 下单 + 05/06 支付 + 09 生命周期，这是钱货闭环。
- 做营销的：08 营销 + 13 满减引擎 + 14 拼团 + 15 预售，看预算与防刷。
- **SRE / 架构**：07 库存 + 10 流量 + 11 一致性，三份是系统抗压的底座。
- 面试备战：每份末尾的 Q&A 两页 + 本份”硬骨头 4 连”。

不必从头读到尾。先用本份建立地图，再按你的角色挑 3~4 份深读，读完任意一份都能回到这张全景图定位自己在哪。

docs/slides/ 共 15 份 + 本总览；*docs/blog/* 同主题深度补充

这套架构的一句话总结

核心链路（扣库存 + 落单 + 收钱）必须稳如磐石，周边能力（优惠 / 通知 / 搜索 / 关单）全部可降级。

顺序即设计：

- 能在事务外做的绝不进事务——满减、库存预扣。
- 进了事务就保证原子——订单 + 优惠预算 + outbox 事件。
- 事务外的副作用必须能补偿——库存释放、延迟关单。

internal/order/service.go:63-178 OrderCreate —函数浓缩全部原则

Q1: 为什么库存预扣放 Redis 而不放 DB 事务里？

A: DB 行锁在高并发下排队、尾延迟爆炸；Redis 两桶 Lua 一次 RTT 原子完成，下单链路不被锁拖住。失败由 ReleaseReservation 补偿 + 对账兜底。

Q2: 下单为什么要 outbox，不直接发 MQ？

A: 直接发会有双写问题——订单落库与消息投递不在一个事务，中间崩溃就丢事件。outbox 把事件写进同一个 DB 事务，再由 publisher 至少一次投递。

Q3: 满减算挂了为什么不直接报错？

A: 优惠是锦上添花，不是下单前提。SLO 承诺保护 happy path，所以算失败、预算耗尽都降级为无折扣，用户照样能成交。

Q4: 事务提交后才发 MQ 延迟关单，万一 MQ 挂了订单永远不关？

A: 双保险。RabbitMQ 延迟队列为主，Redis ZSet (OrderTimeKey, 15min 过期分) 定时扫描兜底，MQ 不可用也能关单释放库存。

Q5: 支付成功后库存是怎么真正减少的？

A: 下单只 reserved (预占)；支付事务改完状态后同步 CommitReservation，把 reserved 真正消掉。未支付被关单则 ReleaseReservation 还回 available。

Q6: 游标分页相比深分页到底强多少？

A: 压测里订单列表游标 + 缓存 58,406 RPS / p95 5ms，旧版深分页 8.3 RPS / p95 15.95s ——吞吐约 7,000×、p95 约 3,200×。

代码位置一览

登录 / 鉴权 *internal/user/service.go:82 UserLogin; pkg/utils/jwt/jwt.go:25 GenerateToken;
middleware/jwt.go:16 AuthMiddleware*

商品 / 购物车 *internal/product/service.go:40 ProductShow; internal/cart/service.go:29 CartCreate*

下单中枢 *internal/order/service.go:63–178 OrderCreate (满减:110 Reserve · 117 Tx · 145 outbox · 159 补
偿 · 172 延迟关单)*

库存两桶 *repository/cache/inventory.go:78 Reserve · 98 Commit · 113 Release*

满减引擎 *internal/promo/service.go:162 CalculateBestDiscount · 234 ApplyDiscountInTx*

Outbox *internal/shared/outbox/repo.go:26 Insert · 44 FetchBatch · 53 MarkSent;
initialize/outbox.go:15 InitOutboxPublisher*

支付 / 履约 *internal/payment/service.go:36 PayDown; internal/order/shipping.go:36 ShipOrder · 73
ConfirmReceive*

状态机 / 关单 *internal/order/state.go:21 orderStateTransitions; internal/order/cancel.go:19
CancelUnpaidOrder*